

React

React

- **Library**: JavaScript library for building **user interfaces** (not a full framework).
- **Created**: 2011 by Facebook → now the most popular front-end JS library.
- **Core Idea**: Build **reusable components** → each contains **markup (HTML)** + **logic (JS)**.
- **Why**: Keeps UI code organized, scalable, and maintainable.

Properties

- **Declarative** → describe what UI should look like; React updates DOM efficiently.
- **Composable** → combine small components like building blocks.
- **Reusable** → components can be reused across app.
- **Scalable** → easy to manage large applications.

Advantages

- Faster UIs → minimal DOM operations.
- Declarative → more predictable, easier to debug.
- Reusable components → reduce code duplication.

React - JSX, Elements & Virtual DOM

- React is tricky at first → concepts are interrelated.
- Key topics: **JSX**, **React elements**, **Virtual DOM**.
- Recommended video: [Introduction to React](#) (watch last 2 min for Virtual DOM explanation).

JSX

- Syntax extension for JS → allows **HTML in JS** and **JS in HTML**.
- Not required, but makes React much easier.
- Babel compiles JSX → plain JavaScript.
- Video: [Introduction to JSX](#).

Code Examples

- **HTML inside JS**

```
JavaScript
1  const element = <h1>Hello, world!</h1>
```

- **JS inside HTML**

```
JavaScript
1  const greeting = "Hello, world!"
2  const element = <h1>{greeting}</h1>
```

- **Multiple elements via map()**

```
JavaScript
1  function ListOfCountries() {
2    let scandinavianCountries = [
3      'Norway', 'Sweden', 'Denmark', 'Iceland', 'Finland'
4    ]
5    return (
```

```

6     <ul>
7       {scandinavianCountries.map(country => {
8         return <li>{country}</li>
9       })}
10    </ul>
11  )
12 }

```

React Elements

- **Definition:** Fundamental building blocks of React apps → describe the **UI (what appears on screen)**.
- **Important:** You never write raw HTML → JSX compiles into **JS objects (React elements)**.
- **JSX → React element:** JSX is just **syntactic sugar** for `React.createElement()`.

Code Example - createElement()

```

1  // JSX
2  const element = <h1>Hello, world!</h1>
3
4  // Transpiled to React element
5  const element = React.createElement('h1', {}, 'Hello, world!')
6

```

Properties

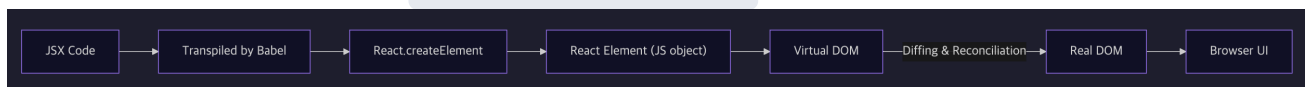
- **Immutable** → once created, React elements **cannot be changed**.
- UI updates = React **discards old elements** and **creates new ones**.
- JSX hides this complexity, letting us write "HTML-like" code while React handles the element creation.

React Virtual DOM

- **Browser DOM:** Browser keeps a **tree representation** of webpage = **Document Object Model (DOM)**.
- **Normal JS:** Directly inserts/modifies elements in DOM → updates UI.
- **React:** You never touch the real DOM directly. Instead, you work with the **Virtual DOM**:
 - You create, delete, and render **React Elements**.
 - React keeps its **Virtual DOM** in sync with the **real DOM**.
 - Updates are optimized → only changed parts of the DOM are re-rendered.

Why Virtual DOM matters

- DOM operations = **slow**.
- React minimizes changes by comparing **previous Virtual DOM** with **new Virtual DOM** → applies minimal updates.
- You're not writing HTML or raw JS → JSX transpiles into **React Elements**.
- Avoid direct DOM manipulation (e.g., `element.innerHTML = "..."`) → breaks React's optimized rendering.



Optional:

Hello World (React docs)

- Use **ReactDOM.createRoot(...)** to target a DOM container.
- Call `root.render(<MyElement />)` to render a React element.
- Elements & components = building blocks of React.
- React keeps a tree of elements and updates the DOM accordingly. legacy.reactjs.org

Introducing JSX

- JSX = syntax extension → embed HTML-like syntax into JavaScript. legacy.reactjs.org
- In JSX:
 - Use **curly braces** `{ }` to embed JavaScript expressions.
 - Attributes with expressions use `{ ... }`, string literals use quotes.
 - `class` → `className`, `tabindex` → `tabIndex`, etc.
- JSX compiles into `React.createElement(type, props, children)` calls.
 - Example:

```
<h1 className="greeting">Hello, world!</h1> // becomes React.createElement('h1', {
  className: 'greeting' }, 'Hello, world!')
```
 - These create **React element objects**. legacy.reactjs.org

Rendering Elements

- React elements = **plain objects**, cheap to create. legacy.reactjs.org
- To render:
 - Get a root DOM container: `const root = ReactDOM.createRoot(...)`
 - Call `root.render(element)` with your React element.
- Updating UI:
 - React elements are **immutable** → to change UI, you create new elements and call `render` again.
 - React compares new element tree with previous → updates **only what's necessary** in real DOM. legacy.reactjs.org

React Components

- **Concept**: Like JavaScript functions → take inputs (`props`) and return **React elements**.
- **Props**:
 - Pass values down to child components.
 - Can be text, booleans, numbers, objects, or even functions.
 - Allow parent ↔ child communication.
- **Composition**: Components can **include other components**.

Class vs. Function Components

- **Class components** (older style, pre-2019):
 - Used ES6 `class` syntax.
 - Had more features (state, lifecycle methods).
- **Functional components** (modern style, recommended):
 - Use simple JavaScript functions.
 - Since React 16.6 → support **hooks** (state + side effects).
 - Cleaner, less verbose, easier to read.
- Both ways produce the same result → important to recognize both (older tutorials still use classes).

Code examples

Function component

```
JavaScript
1 function Welcome() {
2   return <h1>Hello world</h1>
3 }
```

Arrow function version

```
JavaScript
```

```
1 const Welcome = () => <h1>Hello world</h1>
```

Class component

```
JavaScript
1 class Welcome extends React.Component {
2   render() {
3     return <h1>Hello world</h1>
4   }
5 }
```

Reference: [React Components and Props \(legacy docs\)](#)

Video: [Class vs Functional Components](#)

Props

- **Definition:** Short for “properties” → inputs to components (like arguments to functions).
- **Purpose:** Allow **data flow** from parent → child components.
- **Usage:**
 - Pass **text** → e.g. `<Welcome name="Paul" />`.
 - Pass **booleans** → toggle elements on/off.
 - Pass **functions** → allow child to call back and affect parent.
- **Immutability:**
 - Props are **read-only**.
 - Cannot be modified by child components.
 - If parent changes props → child re-renders with updated values.

Creating Props (in parent)

You pass props like you would attributes on an HTML element:

```
JavaScript
1 // Parent component
2 function App() {
3   return (
4     <div>
5       <Welcome name="Paul" age={30} />
6     </div>
7   )
8 }
```

- `name="Paul"` → string prop
- `age={30}` → number prop (curly braces = JS values)

Referencing Props (in child)

Inside the child component, you access them via the `props` object:

```
JavaScript
1 // Child component
2 function Welcome(props) {
3   return <h1>Hello, {props.name}. You are {props.age} years old.</h1>
4 }
5 // OR
6 function Welcome({ name, age }) {
7   return <h1>Hello, {name}. You are {age} years old.</h1>
8 }
```

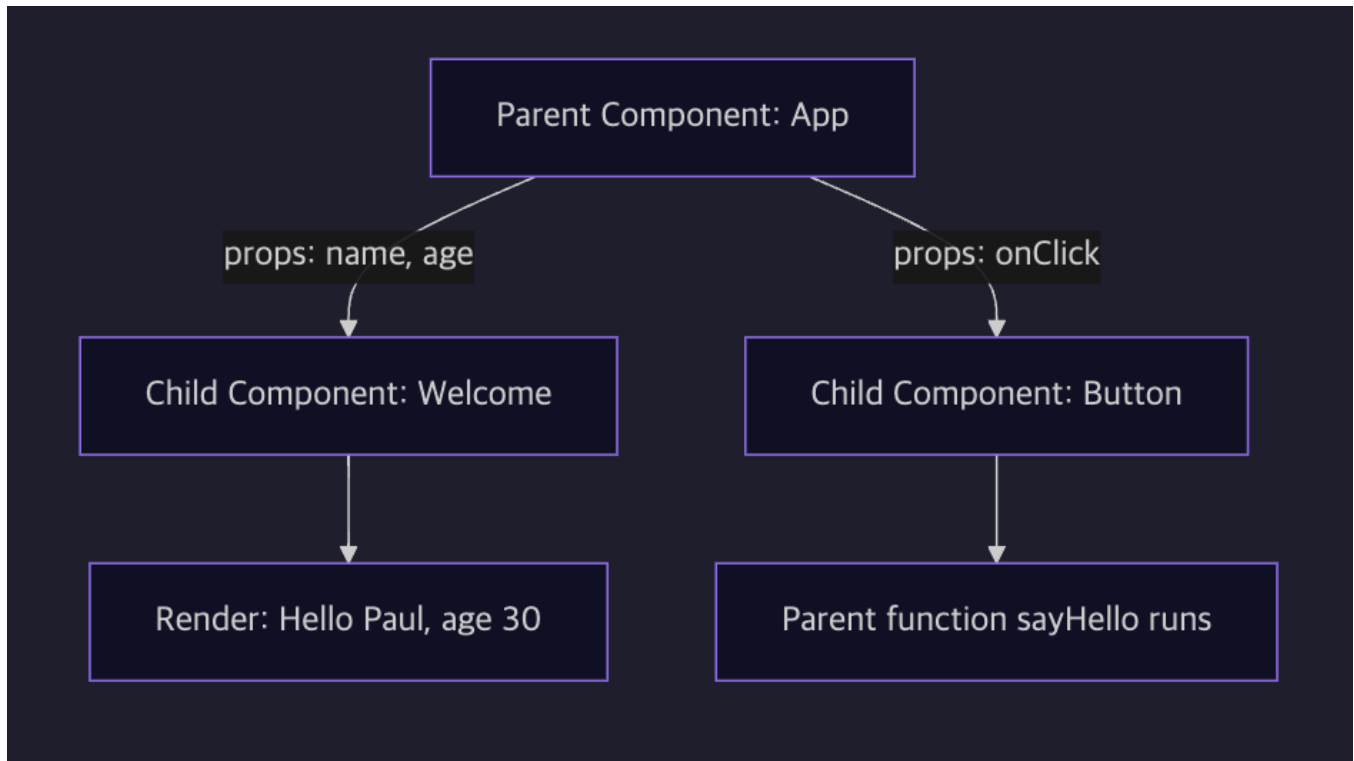
Function props

```
JavaScript
```

```

1 function App() {
2   const sayHello = () => alert("Hello from parent!")
3
4   return <Button onClick={sayHello} />
5 }
6
7 function Button({ onClick }) {
8   return <button onClick={onClick}>Click me</button>
9 }

```



Key idea

- Components **look like functions**.
- **Props = arguments**.
- Always **one-way data flow** (parent → child).

Reference:

- [Rendering Elements \(React docs\)](#)
- [Components & Props \(React docs\)](#)

State

- **Definition:** Component-specific data that can change over time.
- **Purpose:** Lets components respond to **user input**, **network responses**, or **internal logic** by re-rendering when the state changes.
- **Difference from props:**
 - **Props** → passed **into** a component, controlled by parent.
 - **State** → lives **inside** a component, controlled by itself.
- **Re-rendering:** Any update to state triggers the component (and children depending on it) to re-render.

```

React JSX
1 import React, { useState } from "react";
2
3 function Counter() {
4   // Declare state variable "count" with initial value 0
5   const [count, setCount] = useState(0);
6

```

```

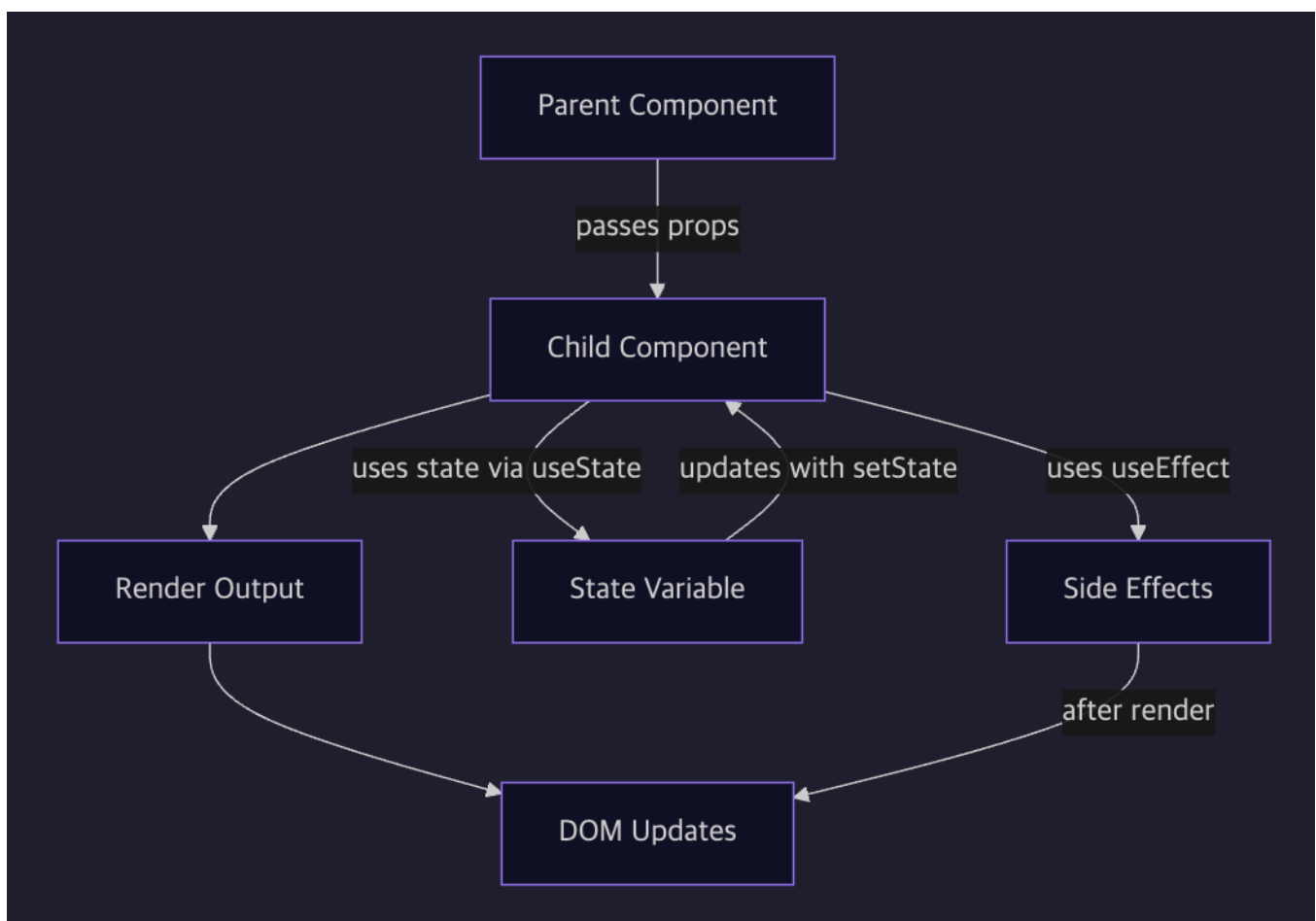
7   return (
8     <div>
9       <p>You clicked {count} times</p>
10      <button onClick={() => setCount(count + 1)}>+1</button>
11      <button onClick={() => setCount(count - 1)}>-1</button>
12    </div>
13  );
14 }

```

- `useState(0)` → initializes state variable `count` to `0`.
- `setCount` → updates the state, causing re-render.
- Buttons update the state by calling `setCount`.

Key points

- State is **mutable**, unlike props.
- State belongs to the **component** that declares it.
- Updating state is **asynchronous** → React batches changes for performance.
- **Never modify state directly** (e.g., `count++`) → always use the setter (`setCount`).



- **Props**: Flow **from parent → child**, read-only inside child.
- **State**: Lives **inside the component**, updated via `useState` or `setState`.
- **Re-render**: Updating state triggers a re-render of that component and its children.

Hooks

- Functions that let you "hook" into **React state** and **lifecycle features** inside function components.
- Introduced in **React 16.8** → no need for class components.
- Solve problems:
 - Sharing stateful logic between components was hard with classes.
 - State & lifecycle logic can now be reused without changing component hierarchy.

- **Rules:**
 - Only call Hooks at the **top level** (not inside loops, conditions, nested functions).
 - Only call Hooks from **functional components** (not regular JS functions).

useState Hook

- Lets function components have **state**.
- Returns a pair: `[value, setValue]`.
- Initial value passed to `useState(initialValue)`.
- Updating with `setValue(newValue)` triggers a **re-render**.

```

1  import React, { useState } from 'react';
2
3  function Example() {
4    const [count, setCount] = useState(0);
5
6    return (
7      <div>
8        <p>You clicked {count} times</p>
9        <button onClick={() => setCount(count + 1)}>Click me</button>
10     </div>
11   );
12 }

```

Equivalent **class component**:

```

1  class Example extends React.Component {
2    constructor(props) {
3      super(props);
4      this.state = { count: 0 };
5    }
6
7    render() {
8      return (
9        <div>
10         <p>You clicked {this.state.count} times</p>
11         <button onClick={() => this.setState({ count: this.state.count + 1 })}>
12           Click me
13         </button>
14       </div>
15     );
16   }
17 }

```

useEffect Hook

- Handles **side effects** (data fetching, subscriptions, DOM updates).
- Runs **after render**.
- By default runs on every render. Can be configured with **dependency array**.

```

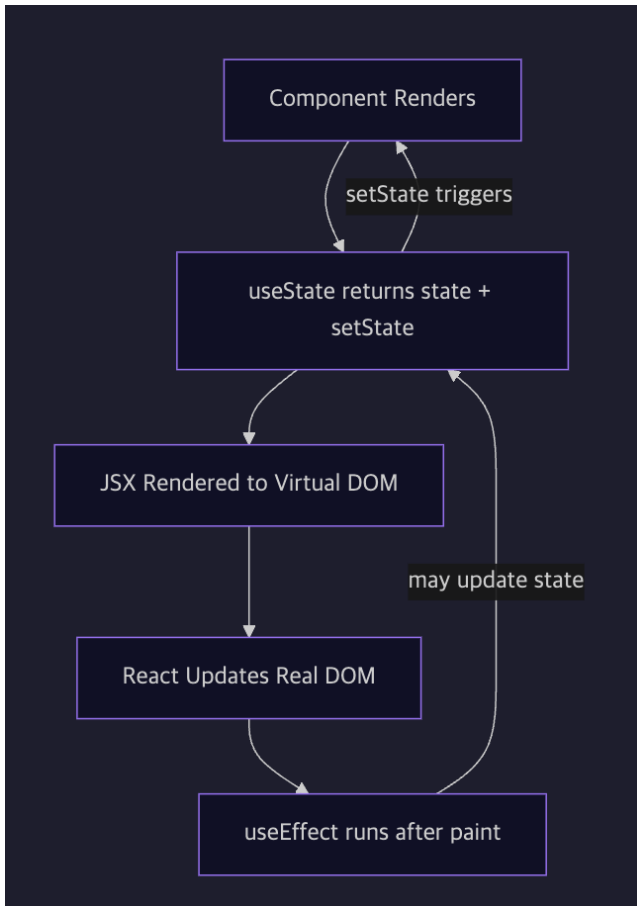
1  import React, { useState, useEffect } from 'react';
2
3  function Example() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      document.title = `You clicked ${count} times`;
8    });
9

```

```

10   return (
11     <div>
12       <p>You clicked {count} times</p>
13       <button onClick={() => setCount(count + 1)}>Click me</button>
14     </div>
15   );
16 }

```



- On first render → `useState` gives you `[state, setState]`.
- React renders JSX → Virtual DOM → updates the real DOM.
- After DOM paint → `useEffect` runs.
- If `useEffect` calls `setState`, the cycle repeats with a **re-render**.